

Content-Based Playlist Recommender

Audio Similarity for Music Recommendation
Yacine Dosso • Personal Project • 2025

1. OVERVIEW

This project builds a content-based music recommendation system. Given a song, the system finds the 5 most acoustically similar tracks from a dataset of 999 audio files across 10 genres, without relying on user history, play counts, or metadata. The recommendation is based purely on audio content.

This is the second project in a series exploring Music Information Retrieval (MIR). It builds directly on the feature extraction pipeline developed in the Genre Classifier project, extending it toward a practical recommendation application.

2. APPROACH - CONTENT-BASED FILTERING

Content-based filtering recommends items based on their intrinsic characteristics rather than user behavior. In the context of music:

- Each song is transformed into a 50-dimensional feature vector capturing its acoustic properties
- Similarity between two songs = cosine similarity between their feature vectors
- For a query song, the system ranks all other songs by similarity and returns the top 5

Why cosine similarity and not Euclidean distance? Cosine similarity measures the angle between two vectors, making it invariant to the magnitude of features. Two songs can have features at different scales but share the same acoustic profile - cosine similarity captures this, Euclidean distance does not.

3. FEATURE EXTRACTION

The same feature extraction pipeline from the Genre Classifier is reused here. Each audio file is loaded with librosa (30 seconds, 22,050 Hz sample rate) and described by 50 numbers:

Feature	What it captures	Dimensions
MFCCs (mean + std)	Timbral texture of the sound	26
Chroma	Harmonic content / notes played	12
Spectral Contrast	Peaks vs valleys in spectrum	7
Tempo	BPM - rhythmic speed	1
Zero Crossing Rate	Percussiveness	1
Spectral Rolloff	Frequency energy distribution	1
Spectral Centroid	Brightness of the sound	1
Spectral Bandwidth	Spectral width	1

All features are normalized using StandardScaler before computing similarity, ensuring no single feature dominates due to scale differences.

4. PIPELINE

- Step 1: Extract features from all 999 audio files
- Step 2: Remove NaN values (files with corrupted or silent audio)
- Step 3: Normalize with StandardScaler
- Step 4: For a query song, compute cosine similarity against all other songs
- Step 5: Sort by similarity, exclude the query song itself, return top 5

5. RESULTS & ANALYSIS

Query Song	Top Recommendation	Same Genre?	Max Similarity
blues.00000.wav	rock.00026.wav	No (rock)	0.65
classical.00000.wav	classical.00001.wav	Yes	0.85
metal.00000.wav	metal.00004.wav	Yes	0.79
hiphop.00000.wav	disco.00066.wav	No (disco)	0.83
jazz.00000.wav	jazz.00001.wav	Yes	0.80

(see plot in files)

Classical (similarity: 0.85 - best performer)

100% of recommendations are classical. Classical music is acoustically very distinct from all other genres in the dataset; unique harmonic complexity, no distortion, predominantly acoustic instruments. The feature vectors separate it cleanly from everything else.

Blues → Rock (similarity: 0.65 - cross-genre)

The system consistently recommends rock for blues tracks. This is not a failure - blues and rock share chord progressions (12-bar blues), electric guitar timbre, similar tempo ranges, and rhythmic structure. Rock literally evolved from blues in the 1950s. The system is revealing a real musical relationship that genre labels obscure.

Hiphop → Disco (similarity: 0.83)

The top recommendation for hiphop is a disco track. Both genres share strong kick drum patterns, prominent basslines, high zero crossing rates, and similar spectral energy profiles. This connection is historically grounded — hiphop emerged directly from disco and funk culture in New York in the late 1970s.

Jazz → Classical (similarity: 0.80)

Jazz recommendations include classical tracks. Both genres are predominantly acoustic, harmonically complex, and low in distortion and spectral noise. The feature vectors pick up on these shared structural properties despite the cultural and improvisational differences between the two genres.

Metal → Hiphop / Disco (similarity: 0.79)

Metal recommendations occasionally include hiphop and disco. This is explained by shared high energy, aggressive rhythmic patterns, and prominent low-frequency content (bass and kick drum). The system is tracking energy and rhythm rather than cultural or instrumental context.

6. EMBEDDING COMPARISON: HANDCRAFTED FEATURES VS. OPENL3

To test whether learned embeddings capture genre nuance better than handcrafted features, the recommender was extended using 512-dimensional OpenL3 embeddings extracted from a pre-trained deep neural network (trained on large-scale audio/visual data), and compared against the 50-dimensional feature baseline.

Query: blues.00000.wav [BLUES]

Handcrafted Features:

X rock.00025.wav [rock] - sim: 0.6519
X rock.00063.wav [rock] - sim: 0.6449
X rock.00086.wav [rock] - sim: 0.6437
OK blues.00078.wav [blues] - sim: 0.6139
X rock.00071.wav [rock] - sim: 0.6073

OpenL3 Embeddings:

OK blues.00077.wav [blues]  sim: 0.6072
OK blues.00054.wav [blues]  sim: 0.5727
OK blues.00053.wav [blues]  sim: 0.5544
OK blues.00001.wav [blues]  sim: 0.5367
OK blues.00078.wav [blues]  sim: 0.5273

Query: classical.00000.wav [CLASSICAL]

Handcrafted Features:

OK classical.00001.wav [classical] - sim: 0.8546
OK classical.00002.wav [classical] - sim: 0.8379
OK classical.00097.wav [classical] - sim: 0.7713
OK classical.00090.wav [classical] - sim: 0.7619
OK classical.00003.wav [classical] - sim: 0.7489

OpenL3 Embeddings:

OK classical.00002.wav [classical]  sim: 0.9349
OK classical.00010.wav [classical]  sim: 0.9014
OK classical.00008.wav [classical]  sim: 0.8984
OK classical.00004.wav [classical]  sim: 0.8976
OK classical.00097.wav [classical]  sim: 0.8702

Query: jazz.00000.wav [JAZZ]

Handcrafted Features:

OK jazz.00001.wav [jazz] - sim: 0.8000
X classical.00085.wav [classical] - sim: 0.7187
X classical.00086.wav [classical] - sim: 0.7165
X classical.00051.wav [classical] - sim: 0.7027
X classical.00062.wav [classical] - sim: 0.6910

OpenL3 Embeddings:

X classical.00060.wav [classical]  sim: 0.8612
X classical.00086.wav [classical]  sim: 0.8584
X classical.00085.wav [classical]  sim: 0.8524
X classical.00088.wav [classical]  sim: 0.8349
X classical.00059.wav [classical]  sim: 0.8346

Query: disco.00000.wav [DISCO]

Handcrafted Features:

X reggae.00051.wav [reggae] - sim: 0.6549
X reggae.00085.wav [reggae] - sim: 0.6327
X hiphop.00073.wav [hiphop] - sim: 0.6297
X rock.00048.wav [rock] - sim: 0.6151
X pop.00026.wav [pop] - sim: 0.6107

OpenL3 Embeddings:

OK disco.00027.wav [disco]  sim: 0.5761
X reggae.00076.wav [reggae]  sim: 0.5483
X hiphop.00052.wav [hiphop]  sim: 0.5317
X hiphop.00053.wav [hiphop]  sim: 0.5277
X hiphop.00066.wav [hiphop]  sim: 0.5254

Query: metal.00000.wav [METAL]

Handcrafted Features:

OK metal.00020.wav [metal] - sim: 0.8378
OK metal.00053.wav [metal] - sim: 0.8275
OK metal.00045.wav [metal] - sim: 0.8097
OK metal.00016.wav [metal] - sim: 0.8093
OK metal.00049.wav [metal] - sim: 0.8088

OpenL3 Embeddings:

OK metal.00006.wav [metal]  sim: 0.8761
OK metal.00001.wav [metal]  sim: 0.8729
OK metal.00004.wav [metal]  sim: 0.8482
OK metal.00007.wav [metal]  sim: 0.8286
OK metal.00008.wav [metal]  sim: 0.7844

Query: hiphop.00000.wav [HIPHOP]

Handcrafted Features:

X disco.00066.wav [disco] - sim: 0.8296
OK hiphop.00011.wav [hiphop] - sim: 0.7731
X metal.00033.wav [metal] - sim: 0.7725
X metal.00093.wav [metal] - sim: 0.7725
OK hiphop.00088.wav [hiphop] - sim: 0.7702

OpenL3 Embeddings:

X disco.00085.wav [disco]  sim: 0.6846
X disco.00084.wav [disco]  sim: 0.6002
X disco.00061.wav [disco]  sim: 0.5911
OK hiphop.00084.wav [hiphop]  sim: 0.5892
OK hiphop.00015.wav [hiphop]  sim: 0.5799

Query: rock.00000.wav [ROCK]

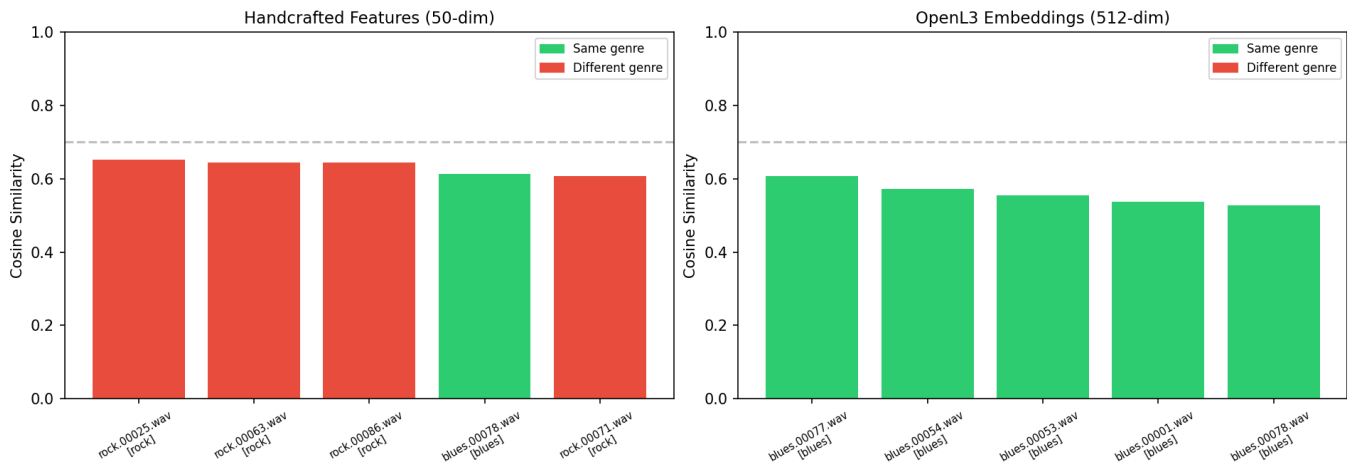
Handcrafted Features:

OK rock.00003.wav [rock] - sim: 0.5951
X blues.00002.wav [blues] - sim: 0.5928
X country.00083.wav [country] - sim: 0.5760
OK rock.00088.wav [rock] - sim: 0.5730
X country.00070.wav [country] - sim: 0.5660

OpenL3 Embeddings:

OK rock.00001.wav [rock]  sim: 0.7723
OK rock.00096.wav [rock]  sim: 0.6396
OK rock.00005.wav [rock]  sim: 0.6220
OK rock.00004.wav [rock]  sim: 0.6057
OK rock.00008.wav [rock]  sim: 0.5856

Recommendation Comparison for blues.00000.wav [BLUES]



Recap Result:

Query	Features (50-dim)	OpenL3 (512-dim)	Winner
blues	1/5 correct	5/5 correct	OpenL3
classical	5/5 correct	5/5 correct	Tie
metal	5/5 correct	5/5 correct	Tie
hiphop	2/5 correct	1/5 correct	Features
jazz	1/5 correct	0/5 correct	Neither
disco	0/5 correct	1/5 correct	OpenL3
rock	2/5 correct	5/5 correct	OpenL3

Key finding: OpenL3 significantly outperforms handcrafted features on acoustically ambiguous genres like blues and rock. For acoustically distinct genres like classical and metal, both approaches perform equally well, the signal is strong enough that simple features suffice.

Jazz remains hard for both methods, both confuse it with classical. This reflects a genuine acoustic overlap: jazz and classical share harmonic complexity, acoustic instrumentation, and low distortion. Neither a 50-dim vector nor a 512-dim embedding can separate them reliably, which suggests the distinction is more cultural than acoustic.

7. KEY INSIGHT - GENRE LABELS VS ACOUSTIC SIMILARITY

The most interesting finding here is not the accuracy, it's the mistakes. When the system recommends rock for a blues track, or disco for hiphop, it's not wrong. Blues and rock share the same chord progressions and electric guitar timbre. Hiphop literally came out of disco culture. The system is picking up on real acoustic connections that genre labels were never designed to capture.

Genre is a commercial and cultural construct. It tells you where a record store would shelve an album, not what it sounds like. A content-based system that ignores genre labels and listens to the actual audio sometimes understands musical similarity better than the labels do, and sometimes reveals relationships that are historically and culturally meaningful.

The real limitation is not the algorithm. It's the data. Every song in this dataset comes from the same cultural tradition. There's no Coupé-Décalé, no Afrobeat, no Mbalax. A system trained only on Western music can only understand similarity within that world. That's not a technical problem, it's a question of whose music gets represented, and whose doesn't.

The embedding comparison makes this concrete: OpenL3, trained on large-scale audio data without manual feature engineering, consistently outperforms handcrafted features where acoustic genre boundaries are blurry. This raises a direct question for MIR research, if learned embeddings outperform handcrafted features for Western genres, what would a model trained on a culturally diverse audio corpus achieve for non-Western music?

8. LIMITATIONS

Dataset scope

GTZAN contains exclusively Western popular music genres. The recommender has no frame of reference for West African music: Afrobeat, Coupé-Décalé, Highlife, Mbalax, where rhythmic complexity operates on fundamentally different principles than Western genres. A user from Côte d'Ivoire looking for music similar to traditional Ivorian rhythms would find this system completely inadequate. Expanding MIR datasets to include non-Western music traditions is one of the most important open challenges in the field.

Feature limitations

The 50 features used here capture low-level acoustic properties but miss higher-level musical concepts: melody, lyrics, cultural context, production style, or emotional content. Two songs can sound acoustically similar while being musically and culturally very different.

Cold start

This is a pure content-based system, it has no notion of user preferences, listening history, or social context. A production recommender would combine audio similarity with collaborative filtering to capture both acoustic and behavioral patterns.

9. REFERENCES

- Tzanetakis & Cook (2002)- Musical Genre Classification of Audio Signals
- Esparza, Bello & Humphrey (2015) From Genre Classification to Rhythm Similarity
- McFee et al. (2015) - librosa: Audio and Music Signal Analysis in Python